

PhySDV: A Physics-Grounded Benchmark for Risk-Aware Perception Offloading in Software-Defined Vehicles

Konstantinos Gyftodimos, Eirini Liotou, Eleni Politi, Kyriakos Chiotis, George Dimitrakopoulos
Dept. of Informatics and Telematics, Harokopio University of Athens (HUA), Athens, Greece
kgyftodimos@anadelta.hua.gr, {eliotou, politie, kchiotis, gdimitra}@hua.gr

Abstract—Software-defined vehicles (SDVs) decouple automotive functions from fixed electronic control units, turning perception into a schedulable service that may execute on board, on a multi-access edge computing (MEC) server, or in the cloud. Selecting the execution tier per frame requires anticipating network conditions that change on sub-second timescales, yet open, labeled datasets coupling vehicular radio state to offloading targets remain scarce. We present PhySDV, an open end-to-end framework comprising (i) a physics-grounded generative model (3GPP-calibrated path loss, shadowing, Shannon-capacity throughput, queue-theoretic server delay) emitting a 50 000-sample labeled dataset across five road archetypes; (ii) a two-stage learned network-state predictor extended with an upper-quantile variant for *risk-aware* feasibility gating; and (iii) a QoS-constrained edge-first decision layer with hysteresis-based stabilization. We further introduce a three-level validation protocol (structural, predictive, task-level): the generator’s configured physics are recoverable from the emitted data to within 1.6%; a gradient-boosted predictor attains 6.1 ms MEC-RTT mean absolute error in a 1.3 MB model; and in closed loop the learned policy tracks a ground-truth oracle to within QoS-neutral tier swaps (up to 98.8% raw decision agreement). Quantile gating raises worst-scenario latency-budget compliance from 94.0% to 96.6% at unchanged effective accuracy, while stabilization suppresses 96% of mode oscillations without delaying safety fallbacks. Framework, dataset, and all experiments are released as open source.

Index Terms—software-defined vehicles, multi-access edge computing, computation offloading, synthetic datasets, quality of service, quantile regression

I. INTRODUCTION

The software-defined vehicle (SDV) paradigm replaces dozens of function-specific electronic control units with a zonal architecture built around centralized, updatable high-performance computers, exposing automotive functions as orchestrated services [1]. This shift makes workload placement a runtime software decision rather than a wiring-harness decision: a perception service such as YOLOv8 object detection [2] may run on an on-board accelerator, be offloaded to a road-side multi-access edge computing (MEC) server [3], or be shipped to a cloud data center—and the placement can be revised per camera frame. Each tier trades accuracy against latency: compact on-board models respond in milliseconds but lose accuracy; larger remote models pay a volatile network round-trip time (RTT). Advanced driving assistance imposes hard quality-of-service (QoS) budgets—end-to-end latencies on the order of 100 ms [4]—so the tier decision must be re-evaluated continuously as radio conditions evolve [5].

Making this decision *predictively* requires estimating near-future MEC and cloud RTT from observable vehicle and

radio state. Data-driven network models [6], [7] are natural candidates but need labeled training data, and public vehicular datasets coupling radio state to offloading-relevant targets (per-tier RTT, utilization, reachability) are scarce: operator traces are proprietary, while packet-level co-simulators [8], [9] do not emit supervised-learning-ready labels and are heavyweight to configure and run.

This paper closes that gap with **PhySDV**, an open, lightweight, fully reproducible framework¹ that spans the complete loop from radio physics to per-frame offloading decisions. Our contributions are:

- 1) **A physics-grounded generative model and open dataset.** A 3GPP-calibrated log-distance channel [10] with log-normal shadowing and stochastic non-line-of-sight (NLOS) excursions, composed with Shannon-capacity throughput [11], per-user bandwidth sharing, queue-theoretic server delay [12], and a diurnal load model. The released dataset covers 50 000 samples across five road archetypes with MEC inter-site distances from 300 m to 10 km (Sec. IV).
- 2) **A deployable two-stage network-state predictor.** An availability classifier gates a multi-output RTT regressor, avoiding sentinel-value contamination. A model-family study identifies gradient-boosted trees as the deployment choice: 6.1 ms MEC-RTT MAE in a 1.3 MB model—700× smaller and 3.8× faster than the random-forest baseline (Sec. V).
- 3) **Risk-aware QoS gating via quantile regression.** Feasibility is gated on the predicted *upper quantile* (p90) of RTT rather than its mean, converting symmetric error into a one-sided safety margin and closing most of the compliance gap to the oracle (Secs. VI, VIII).
- 4) **A stabilized, explainable decision layer.** Edge-first selection with exponential smoothing and minimum-dwell hysteresis (safety bypass on infeasibility) suppresses 96% of mode oscillations; every decision carries a machine-readable reason code for the auditability SDV safety cases require (Sec. VI).
- 5) **A three-level validation protocol** for synthetic network datasets: *structural* (configured physics recoverable from the data), *predictive* (held-out error), and *task-level* fidelity (closed-loop agreement with a ground-truth oracle) (Sec. VII).

¹<https://github.com/icsa-hua/PHYSDV>

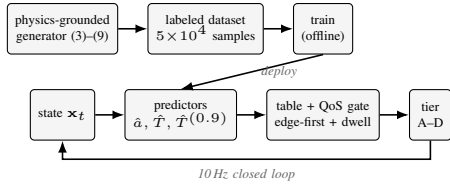


Fig. 1. PhySDV architecture: offline data generation and training; online per-frame prediction, QoS gating, and stabilized tier binding.

Fig. 1 summarizes the architecture: PhySDV is a *framework*, not a one-off dataset. Offline, the generator emits labeled samples on which the predictor stack is trained; online, the predictors turn the observable state $\mathbf{x}_t$ into per-tier estimates that a QoS-gated, stabilized policy binds to an execution tier at frame rate. Every stage is replaceable: the released dataset is one seeded draw of the generator, alternative predictors or policies drop into the same loop, and the validation protocol applies unchanged.

The remainder of the paper is structured as follows. Sec. II reviews related work; Sec. III formalizes the problem; Secs. IV–VI present the generator, predictor, and decision layer; Sec. VII defines the validation protocol; Sec. VIII reports results; Sec. IX concludes.

II. RELATED WORK

Computation-offloading policy design is extensively surveyed in [13], [14]. Learning-based policies range from deep reinforcement learning (DRL) over abstracted channel models [15] to DNN partitioning between device and cloud [16]. These works optimize the *policy* while abstracting the network as a stationary or coarsely modeled resource; PhySDV is complementary, supplying the missing substrate: a reproducible, physically consistent environment with labeled network state on which such policies can be trained and compared.

In the vehicular domain, edge computing is a recognized enabler for automotive perception [5], and the SDV transition makes workload placement a first-class software concern [1]. Prior evaluations typically rely on proprietary drive traces or on packet-level co-simulation stacks coupling network and traffic simulators [8], [9], which reproduce protocol dynamics faithfully but require expert configuration, run far slower than real time, and do not natively emit supervised-learning labels. PhySDV trades packet-level detail for an analytically transparent link/queue model that generates labeled samples in microseconds, is seeded end-to-end, and runs on a laptop.

A complementary line releases *measured* vehicular network datasets: Raca *et al.* publish 4G/5G throughput traces with channel and context metrics [17], and Lumos5G maps commercial mmWave-5G throughput [18]. Such traces capture real radio dynamics but are fixed observational records: they log client-side throughput rather than the per-tier RTT, utilization, and reachability labels an offloading policy must predict, and cannot be re-parameterized. PhySDV is generative—labels are complete and physics-consistent by construction—and is designed to be *calibrated* against exactly such traces (Sec. VIII).

TABLE I
EXECUTION-TIER PROFILES (AS CONFIGURED IN THE FRAMEWORK)

Tier	HW / model	Inf. (ms)	Net. (ms)	mAP	Δ_m
A	NPU / YOLOv8n	8	0	37.3	20.8%*
B	GPU / YOLOv8n	5	0	37.3	20.8%*
C	MEC / YOLOv8m	6	$\hat{T}_C(t)$	50.2	4.9%
D	Cloud / YOLOv8l	4	$\hat{T}_D(t)$	52.9	0.0%

*Exceeds $\Delta_{\max} = 15\%$: activated only as emergency fallback.

Closer to our predictive layer, RouteNet [6] and the network digital-twin agenda [7] show that learned surrogates predict network performance accurately—given training data. We operationalize this idea for the vehicular access segment and add two missing elements: a validation protocol that checks the *data generator itself* (Sec. VII), and a risk-aware quantile formulation [22] that converts the surrogate’s residual uncertainty into a QoS safety margin.

III. PROBLEM STATEMENT

A. Four-tier execution hierarchy

We model perception as an SDV service: the vehicle streams camera frames at rate f (default 10 Hz) to an object detector, and an on-board orchestrator binds each frame t to exactly one execution tier $m \in \mathcal{M} = \{A, B, C, D\}$: (A) an on-board neural processing unit and (B) the primary on-board GPU, both running the compact YOLOv8n; (C) a MEC server running YOLOv8m; and (D) a cloud data center running YOLOv8l (Table I). Tier profiles are configurable constants; shipped values use published COCO [19] mean average precision (mAP) figures for YOLOv8 [2], and a benchmark module can substitute latencies measured on real hardware.

The *accuracy loss* Δ_m of tier m is defined against the strongest model (YOLOv8l) over the safety-relevant road classes $\mathcal{C} = \{\text{person, car, motorcycle, bus, truck}\}$:

$$\Delta_m = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \frac{\text{AP}_c^{\text{ref}} - \text{AP}_c^m}{\text{AP}_c^{\text{ref}}}, \quad (1)$$

using per-class average precision (AP). Class-weighted evaluation matters: the compact model’s AP drop is far from uniform (28.1% on *truck* vs. 15.7% on *person*), which a single aggregate mAP delta would mask.

B. QoS constraints and decision problem

Let $T_m(t)$ denote the true end-to-end latency of tier m at frame t (inference plus network RTT; zero network term for A/B), $U_m(t)$ the serving node’s utilization, and $a(t) \in \{0, 1\}$ MEC reachability. The QoS contract imposes three hard budgets: $L_{\max} = 100$ ms end-to-end latency [4], $\Delta_{\max} = 15\%$ accuracy loss, and $U_{\max} = 95\%$ utilization. The feasible set at frame t is

$$\mathcal{F}_t = \left\{ m : \rho_m(t) = 1, T_m(t) \leq L_{\max}, \Delta_m \leq \Delta_{\max}, U_m(t) < U_{\max} \right\}, \quad (2)$$

where $\rho_m(t)$ is reachability ($\rho_C = a$, $\rho_m = 1$ otherwise). The system must choose m_t before executing frame t , using only

predictions $\hat{T}_m(t), \hat{U}_m(t), \hat{a}(t)$ derived from the observable state $\mathbf{x}_t$ (Sec. IV); mispredictions that admit an infeasible tier translate directly into QoS violations, motivating both the learned predictor (Sec. V) and the risk-aware gate (Sec. VI).

IV. PHYSICS-GROUNDED DATA GENERATION

The generator is the root of the framework in Fig. 1: it encodes the causal chain from deployment geometry and radio physics to the latencies a policy experiences, with three design goals: transparency (every constant is a named parameter), speed (a sample costs microseconds), and label completeness. Concretely, each sample couples a 13-dimensional observable state $\mathbf{x}$ (scenario class, vehicle speed, time of day, MEC distance, received signal strength, channel bandwidth, cell load, connected users, handover flag, payload size, wide-area-network (WAN) congestion, base cloud RTT, cloud load) to five targets: MEC reachability a , MEC and cloud RTT, and MEC and cloud utilization. Targets are *derived*, not sampled: every label follows deterministically (up to bounded measurement noise) from the physics below, which is what makes structural validation (Sec. VII) possible.

A. Radio link

Path loss follows a 3GPP-style log-distance model [10] with scenario-specific exponent n :

$$\text{PL}(d) = 32.4 + 10n \log_{10} d + 20 \log_{10} f_c, \quad (3)$$

with d the MEC distance (m) and f_c the carrier frequency (GHz). Received signal strength adds log-normal shadowing and stochastic NLOS excursions:

$$P_{\text{rx}} = P_{\text{tx}} + G - \text{PL}(d) + X_\sigma - \mathbf{1}_{\text{NLOS}} \mathcal{U}(25, 45) \text{ dB}, \quad (4)$$

where $X_\sigma \sim \mathcal{N}(0, \sigma^2)$ and NLOS occurs with scenario probability p_{NLOS} . With orthogonal intra-cell scheduling, the signal-to-interference-plus-noise ratio (SINR) is

$$\gamma = P_{\text{rx}} - (-174 + 10 \log_{10} B + N_F) - I_{\text{ic}}, \quad (5)$$

with bandwidth B (Hz), noise figure $N_F = 7$ dB, and a fixed inter-cell interference margin $I_{\text{ic}} = 5$ dB. MEC reachability is the link-budget test $a = \mathbf{1}[\gamma > 0 \text{ dB}]$.

B. Throughput and per-tier latency

Spectral efficiency follows Shannon capacity [11] with an implementation factor $\kappa = 0.6$ and a 256-QAM cap:

$$\eta_{\text{se}} = \min(\kappa \log_2(1 + 10^{7/10}), 8.0) \text{ bit/s/Hz}. \quad (6)$$

The per-vehicle uplink rate shares the channel among N_{UE} users and degrades with server load ρ_{mec} :

$$R = \eta_{\text{se}} \cdot \frac{B}{N_{\text{UE}}} \cdot (1 - 0.7 \rho_{\text{mec}}). \quad (7)$$

MEC RTT composes upload, propagation, queueing, and handover terms:

$$T_{\text{mec}} = \frac{8S}{R} + \frac{2d}{\nu} + \beta \rho_{\text{mec}}^2 + T_{\text{ho}} \mathbf{1}_{\text{ho}}, \quad (8)$$

with payload S (kB), propagation speed $\nu = 2 \times 10^8$ m/s, a queue-theoretic [12] load-quadratic delay ($\beta = 15$ ms), and

TABLE II
ROAD-SCENARIO PROFILES (ISD = INTER-SITE DISTANCE)

Scenario	ISD (m)	Speed (km/h)	f_c (GHz)	B (MHz)	n	$\bar{N}_{\text{UE}}$
urban_dense	300	20–50	3.5	100	3.5	40
urban_sparse	800	30–70	3.5	100	3.0	20
suburban	2000	50–90	2.1	40	2.6	10
highway	5000	80–130	2.1	20	2.2	5
rural	10000	60–100	0.9	10	2.0	2

$T_{\text{ho}} = 20$ ms per handover. Cloud RTT adds a WAN model with congestion index $g \in [0, 1]$:

$$T_{\text{cloud}} = T_{\text{base}} + \frac{8S}{B_{\text{wan}}/(1 + 3g)} + 30g. \quad (9)$$

Both RTTs receive bounded multiplicative measurement noise ($\pm 10\%$ MEC, $\pm 5\%$ cloud). Handover probability rises to 0.30 near the cell edge at speed, cell load follows a Beta distribution with a $1.5\times$ diurnal rush-hour factor, and user count is Poisson with scenario-specific mean—giving realistic correlation structure.

C. Road-scenario parameterization

Five archetypes (Table II) span two orders of magnitude in MEC inter-site distance and the corresponding spectrum allocations, from a dense 3.5 GHz urban micro-cell layout to a 900 MHz rural macro cell.

V. LEARNED NETWORK-STATE PREDICTOR

A. Two-stage formulation

One structural obstacle shapes the predictor: unreachable frames have no finite MEC RTT; encoding them with a sentinel value corrupts regression training, while discarding them biases the classifier. PhySDV therefore factorizes the predictor: a binary classifier estimates $\hat{a}(t)$, and a multi-output regressor estimates RTT and utilization; at inference time the regressor's MEC output is masked whenever $\hat{a} = 0$. Both stages share one pipeline: one-hot scenario encoding, standardization, and cyclic time-of-day encoding $(\sin, \cos)(2\pi h/24)$, removing the artificial midnight discontinuity. Utilization targets are near-pass-through in the current generator and retained for schema completeness; the substantive learning problem is RTT prediction, on which we focus.

B. Model-family study

Deployment constraints—model size, single-frame latency, retraining cost—matter as much as accuracy on an automotive platform. We compare five families under identical preprocessing and an 80/20 split, all implemented in scikit-learn [23] (Table III). MEC-RTT metrics are computed *only on reachable test rows*, the operationally meaningful regime. A linear model fails outright on MEC RTT ($R^2 < 0$): the target is dominated by the nonlinear interaction of link budget, bandwidth share, and load. Bagged trees (the original default) model MEC RTT well but regress the smooth WAN target poorly and serialize to 926 MB. Histogram gradient boosting [21] dominates both

TABLE III
PREDICTOR COMPARISON (80/20 SPLIT; MEC RTT SCORED ON REACHABLE ROWS). † = FRAMEWORK DEFAULT; BOLD = RECOMMENDED.

Model	MEC RTT		Cloud RTT		Lat. (ms)	Size
	MAE	R ²	MAE	R ²		
Ridge (linear)	122.8	-2.62	1.67	0.975	1.1	3 kB
Random forest [†] [20]	10.5	0.930	4.06	0.810	14.3	926 MB
Extra trees	8.1	0.940	4.29	0.794	14.1	1.43 GB
Hist. grad. boosting	6.1	0.959	1.69	0.974	3.8	1.3 MB
MLP (128-64)	7.1	0.952	1.80	0.970	1.3	0.3 MB

targets—6.12 ms MEC and 1.69 ms cloud MAE—in a 1.3 MB artifact with 3.8 ms latency, and is the recommended deployment predictor. The availability stage is near-perfect for every family (gradient boosting: 100.0%)—expected, since the label is a deterministic link-budget threshold (Sec. IV)—and is a consistency check, not a claim of difficulty.

Impurity-based attribution confirms the physics: received signal strength dominates (91% of split gain), then bandwidth, user count, and scenario—mirroring the causal chain of (3)–(8).

C. Risk-aware quantile variant

A point predictor minimizing squared error is calibrated to the conditional mean, so roughly half of its errors are underestimates; each underestimate near the budget boundary can admit a tier whose *true* latency violates $L_{\max}$. We therefore train upper-quantile RTT regressors with pinball loss [22] at $\alpha = 0.9$ (gradient boosting, MEC model fitted on reachable rows only); held-out coverage is 88.5% (MEC) and 88.1% (cloud) against the 90% target. The decision layer can then gate feasibility on $\hat{T}_m^{(\alpha)}$ instead of $\hat{T}_m$, turning residual model uncertainty into a one-sided safety margin.

VI. QOS-AWARE DECISION LAYER

The decision layer closes the online loop of Fig. 1; Algorithm 1 summarizes it. Predictions are smoothed by an exponential moving average (EMA, $\alpha_s = 0.3$) to damp frame-to-frame jitter. The feasibility gate applies (2) with either the mean or the quantile RTT estimate. Among feasible tiers the policy is *edge-first*: MEC is preferred over cloud unless the cloud model offers more than $\delta = 5$ mAP points, conserving WAN bandwidth; if no tier is feasible, the loop falls back to the on-board GPU. Every decision emits a machine-readable reason code, giving SDV integrators a per-frame audit trail.

Because the feasibility boundary can sit between near-equivalent tiers, raw per-frame decisions oscillate. A minimum-dwell hysteresis filter commits a switch only after the new tier persists for $k = 5$ consecutive frames; infeasibility-triggered fallbacks *bypass* the filter, so safety-critical transitions are never delayed (Sec. VIII).

VII. THREE-LEVEL VALIDATION PROTOCOL

Synthetic datasets are only useful if their fidelity is testable. We validate PhySDV at three levels, each with a falsifiable criterion:

Algorithm 1 PhySDV per-frame offloading decision

```

1: observe state  $\mathbf{x}_t$ ; predict  $\hat{a}$ ,  $\hat{T}_C$ ,  $\hat{T}_D$  (optionally  $\hat{T}_C^{(\alpha)}$ ,  $\hat{T}_D^{(\alpha)}$ ,  $\hat{U}_C$ ,  $\hat{U}_D$ )
2:  $\tilde{T}_m \leftarrow \alpha_s \hat{T}_m + (1 - \alpha_s) \hat{T}_m \triangleright$  EMA; reset on  $\hat{a} = 0$ 
3: build table: tiers A/B static; C/D  $\leftarrow (\tilde{T}_m, \hat{U}_m)$ 
4:  $\mathcal{F}_t \leftarrow$  feasibility gate (2)
5: if  $\mathcal{F}_t = \emptyset$  then
6:    $m_t \leftarrow \mathbf{B}$ ; reason  $\leftarrow$  fallback; force switch
7: else if  $\{C, D\} \subseteq \mathcal{F}_t$  and  $\text{mAP}_D - \text{mAP}_C < \delta$  then
8:    $m_t \leftarrow \mathbf{C} \triangleright$  edge-first
9: else
10:   $m_t \leftarrow \arg \max_{m \in \mathcal{F}_t} \text{mAP}_m$ 
11: end if
12:  $m_t \leftarrow \text{Dwell}_k(m_t) \triangleright$  hysteresis; bypassed on force

```

V1 — Structural fidelity. The generator’s configured physics must be *recoverable* from the emitted data by an independent estimator. We re-fit the path-loss model (3) to the generated (distance, signal-strength) pairs per scenario using iteratively re-weighted least squares (sigma-clipping strips the NLOS mixture), and compare recovered exponents and residual spreads to the configuration.

V2 — Predictive fidelity. Standard held-out metrics (Table III), with the sentinel-aware protocol of Sec. V and quantile-coverage calibration.

V3 — Task-level fidelity. V1–V2 do not establish that predictions are good enough *for the decisions they drive*. We therefore replay the closed loop against ground truth: an *oracle* policy decides with the true $T_m(t)$, $a(t)$, and the learned policy is scored on (i) decision agreement, (ii) true latency-budget compliance, and (iii) effective detection quality. We propose V3—rarely reported in offloading studies—as the primary acceptance criterion for learned offloading stacks.

VIII. EXPERIMENTAL RESULTS

All experiments run CPU-only (Intel Core Ultra 9 275HX, 24 threads, 64 GB RAM), fixed seed 42, on the released 50 000-sample dataset; closed-loop runs simulate 1 000 frames per scenario at 10 Hz.

A. V1: structural fidelity

Table IV shows the recovered path-loss exponents within 0.1–1.6% of their configured values, with clipped residual spread tracking the configured shadowing σ . The diurnal model is likewise recovered: rush-hour vs. off-peak mean cell load is 42.6% vs. 28.6%, a $1.49\times$ ratio against the configured $1.5\times$. Emergent per-scenario structure (Table V) is plausible: availability falls in NLOS-rich dense deployments, and MEC RTT worsens with user density.

A key emergent effect deserves emphasis: in dense urban the *edge is not the faster tier*—MEC RTT beats cloud RTT in only 33% of reachable samples (40 users share the cell; $n = 3.5$ path loss erodes per-user throughput), and p95 MEC RTT exceeds the budget outright. Static “edge always

TABLE IV

V1: RECOVERY OF CONFIGURED PHYSICS FROM THE EMITTED DATA

Scenario	n (conf.)	n (recov.)	σ (conf.)	resid. (dB)
urban_dense	3.5	3.44	8	8.00
urban_sparse	3.0	2.99	8	7.35
suburban	2.6	2.61	6	5.18
highway	2.2	2.19	4	3.35
rural	2.0	2.00	4	3.49

TABLE V

PER-SCENARIO DATASET STATISTICS. “C<D” = SHARE OF REACHABLE SAMPLES WHERE MEC RTT BEATS CLOUD RTT.

Scenario	Avail.	MEC RTT med./p95 (ms)	Cloud RTT med./p95 (ms)	C<D
urban_dense	80.8%	78.7 / 319.3	59.0 / 73.2	33.0%
urban_sparse	83.2%	41.7 / 158.3	64.2 / 81.6	70.9%
suburban	91.4%	39.3 / 128.2	69.7 / 87.2	80.9%
highway	95.9%	34.0 / 110.3	72.4 / 92.2	86.9%
rural	98.7%	29.3 / 133.1	81.0 / 103.0	88.7%

wins” heuristics fail exactly where traffic is densest; per-frame prediction is a requirement, not an optimization.

B. V2: predictive fidelity

Table III (discussed in Sec. V) summarizes held-out accuracy, latency, and footprint; the recommended gradient-boosted predictor reaches 6.1 ms MEC-RTT MAE ($R^2 = 0.959$ on reachable rows) and 1.69 ms cloud MAE, with quantile heads calibrated to within 1.9 points of nominal coverage.

C. V3: closed-loop, task-level fidelity

Table VI compares policies over all five scenarios (5 000 decisions per policy). Static baselines expose the trade-off space: *Always-B* meets latency trivially but violates the accuracy budget on every frame; *Always-D* is accurate but consumes WAN uplink on every frame and drops to 96.6% compliance in the rural archetype; *Always-C* collapses to 53.5% latency compliance in dense urban. The learned policy tracks the oracle closely: mean-gated PhySDV achieves 96.9% compliance and identical effective mAP (50.6) to the oracle while touching the WAN on only 14.6% of frames.

Oracle agreement. Per-frame decision agreement between mean-gated PhySDV and the oracle is 98.5–98.8% in the suburban, highway, and rural archetypes, 89.7% in sparse urban, and 78.6% in dense urban—precisely where Table V shows MEC and cloud in a near-tie. These disagreements are QoS-neutral swaps within the feasible set: dense-urban effective mAP (51.7 vs. 51.5) and mean latency (69.2 vs. 67.7 ms) stay within 3% of the oracle.

Risk-aware gating. Mean-gating leaves a compliance gap to the oracle concentrated in the two urban archetypes (94.6% and 94.0%), exactly where prediction error meets a tight feasibility boundary. Gating on the calibrated p90 estimate closes most of it (Fig. 2): dense-urban compliance rises to 97.0%, sparse-urban to 96.6%, and the five-scenario mean from 96.9% to 97.9%. Effective accuracy is *unchanged* (mAP 50.7): the conservatism is priced in WAN share (+3.3 points to

TABLE VI

V3: CLOSED-LOOP POLICY COMPARISON, MEAN OVER FIVE SCENARIOS (5 000 FRAMES). LAT. / ACC. = LATENCY / ACCURACY-BUDGET COMPLIANCE; $\bar{T}$ = MEAN TRUE LATENCY; WAN = CLOUD SHARE; SW. = SWITCHES PER 1 000 FRAMES.

Policy	Lat.	Acc.	mAP	$\bar{T}$ (ms)	WAN	Sw.
Always-B (on-board)	100.0%	0.0%	37.3	5.0	0%	0
Always-C (MEC)	86.4%	100%	50.2	64.1	0%	0
Always-D (cloud)	99.3%	100%	52.9	73.5	100%	0
PhySDV, no EMA	99.5%	100%	50.6	50.5	14.0%	142
PhySDV (EMA)	96.9%	100%	50.6	51.8	14.6%	47
PhySDV q90 (EMA)	97.9%	100%	50.7	51.0	17.9%	43
PhySDV q90 + dwell	96.3%	100%	50.7	52.8	18.2%	6
Oracle (true state)	100.0%	100%	50.6	50.6	13.7%	139

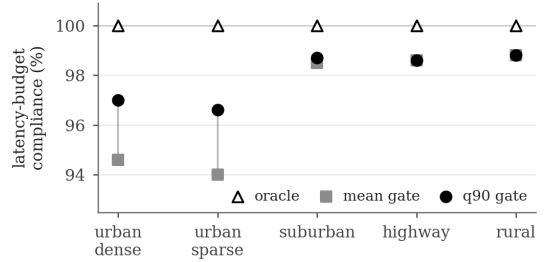


Fig. 2. V3 latency-budget compliance per scenario: gating on the calibrated p90 RTT estimate (q90) recovers most of the mean-to-oracle gap, concentrated in the two urban archetypes.

17.9%) as borderline MEC frames divert to the cloud—making α an explicit, per-deployment dial trading WAN budget for latency assurance.

Stability. Raw per-frame decisions switch 142 times per 1 000 frames; EMA smoothing cuts this to 47; adding the dwell filter yields 6 switches (−96% vs. raw) while mean latency compliance changes by only 1.6 points (97.9% → 96.3%). Since each switch is a service reconfiguration (stream redirection, session renegotiation), this two-order-of-magnitude reduction is an enabler in itself; infeasibility events still bypass the filter instantly.

Outage failover. Fig. 3 replays a dense-urban run with a 3 s connectivity outage injected at frame 80 (MEC unreachable, cloud RTT inflated to 200 ms). Both raw and stabilized policies fall back to the on-board GPU *on the frame of outage onset* (zero-frame delay)—accepting the temporary accuracy-budget violation, which is exactly the intended emergency semantics. The stabilized trace holds the fallback for $k-1 = 4$ extra frames (0.4 s) after the outage clears, a conservative re-entry that avoids flapping.

Runtime. The full control loop—feature assembly, classification, mean and quantile RTT prediction, gating, and stabilization—runs in 10.8 ms median per frame with the recommended predictor, comfortably inside a 10 Hz budget on CPU only. Dataset generation and full training complete in under a minute combined, with no GPU, testbed, or network connection required.

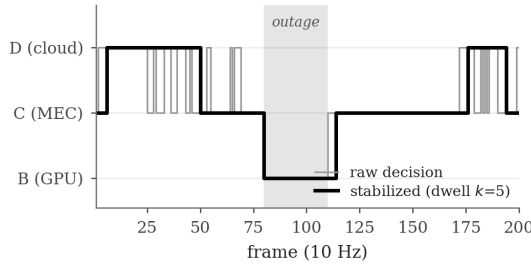


Fig. 3. Outage failover (dense urban, q90-gated). During the injected outage (frames 80–109, shaded) fallback is immediate in both traces because infeasibility bypasses the dwell filter; the stabilized trace also suppresses the C/D chatter.

D. Discussion and limitations

PhySDV’s labels are internally consistent by construction, and V1 shows they faithfully encode the configured physics—but the configuration itself abstracts reality: a single serving cell, no packet-level protocol effects, reachability reduced to a deterministic link budget (real RRC and admission-control behavior is noisier), no multi-vehicle decision coupling, and utilization exposed as in-band telemetry. The intended workflow is *calibration*: every constant in (3)–(9) is a named parameter re-fittable from drive tests such as [17], [18], and the benchmark module replaces configured tier latencies with measured ones; the validation protocol then re-runs unchanged.

These limitations position PhySDV within the landscape of Sec. II: it replaces neither packet-level co-simulation [8], [9] where protocol microdynamics matter, nor measured traces [17] where absolute realism matters; it occupies the gap between them—far lighter than co-simulation, yet label-complete and re-parameterizable where fixed traces are not—the regime in which learned policies [15], [16] iterate quickly.

Nor is the substrate SDV-specific: any latency-sensitive mobile service facing a device/edge/cloud placement choice (teleoperation, drone inspection, mobile XR) maps onto the same state–label schema by swapping tier profiles, and predictor selection, uncertainty calibration, or stability tuning can be studied on the released dataset as-is—regenerable in under a minute from a seeded script. The decision layer is deliberately a transparent, auditable baseline; the framework makes *any* policy comparable under V3, and plugging DRL policies [15] into the loop is direct future work, with multi-vehicle contention and trace-driven calibration.

IX. CONCLUSION

PhySDV contributes an open, physics-grounded substrate for QoS-constrained perception offloading in SDVs: a labeled 50 k-sample dataset whose generating physics are recoverable from the data itself, a deployable two-stage predictor (1.3 MB, 3.8 ms), a risk-aware quantile gate converting prediction uncertainty into QoS safety margin, and a stabilized, explainable decision layer—validated by a three-level protocol culminating in closed-loop oracle agreement of up to 98.8%. We offer PhySDV as a common, extensible baseline on which policies,

predictors, and calibrated deployment profiles can be compared under identical, falsifiable conditions.

ACKNOWLEDGMENT

This work has been funded by the Shift2SDV project (Grant Agreement No. 101194245), supported by the Chips Joint Undertaking and its members, including top-up funding by the national authorities of Austria, Denmark, Germany, Greece, Finland, Italy, Netherlands, Poland, Portugal, Spain, and Turkey.

REFERENCES

- [1] Z. Liu, W. Zhang, and F. Zhao, “Impact, challenges and prospect of software-defined vehicles,” *Automotive Innovation*, vol. 5, pp. 180–194, 2022.
- [2] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” GitHub, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [3] ETSI, “GS MEC 003: MEC framework and reference architecture, v3.1.1,” 2022.
- [4] 3GPP, “TS 22.186: Enhancement of 3GPP support for V2X scenarios, v16.2.0,” 3rd Generation Partnership Project, 2019.
- [5] S. Liu, L. Liu, J. Tang, B. Yu, Y. Wang, and W. Shi, “Edge computing for autonomous driving: Opportunities and challenges,” *Proc. IEEE*, vol. 107, no. 8, pp. 1697–1716, 2019.
- [6] K. Rusek, J. Suárez-Varela, P. Almasan, P. Barlet-Ros, and A. Cabellos-Aparicio, “RouteNet: Leveraging graph neural networks for network modeling and optimization in SDN,” *IEEE J. Sel. Areas Commun.*, vol. 38, no. 10, pp. 2260–2270, 2020.
- [7] P. Almasan *et al.*, “Network digital twin: Context, enabling technologies, and opportunities,” *IEEE Commun. Mag.*, vol. 60, no. 11, pp. 22–27, 2022.
- [8] C. Sommer, R. German, and F. Dressler, “Bidirectionally coupled network and road traffic simulation for improved IVC analysis,” *IEEE Trans. Mobile Comput.*, vol. 10, no. 1, pp. 3–15, 2011.
- [9] P. A. Lopez *et al.*, “Microscopic traffic simulation using SUMO,” in *Proc. IEEE Int. Conf. Intelligent Transportation Systems (ITSC)*, 2018, pp. 2575–2582.
- [10] 3GPP, “TR 38.901: Study on channel model for frequencies from 0.5 to 100 GHz, v16.1.0,” 3rd Generation Partnership Project, 2019.
- [11] C. E. Shannon, “A mathematical theory of communication,” *Bell Syst. Tech. J.*, vol. 27, pp. 379–423, 1948.
- [12] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. New York, NY, USA: Wiley, 1975.
- [13] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, “A survey on mobile edge computing: The communication perspective,” *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, 2017.
- [14] P. Mach and Z. Becvar, “Mobile edge computing: A survey on architecture and computation offloading,” *IEEE Commun. Surveys Tuts.*, vol. 19, no. 3, pp. 1628–1656, 2017.
- [15] X. Chen, H. Zhang, C. Wu, S. Mao, Y. Ji, and M. Bennis, “Optimized computation offloading performance in virtual edge computing systems via deep reinforcement learning,” *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4005–4018, 2019.
- [16] Y. Kang *et al.*, “Neurosurgeon: Collaborative intelligence between the cloud and mobile edge,” in *Proc. ACM ASPLOS*, 2017, pp. 615–629.
- [17] D. Raca, D. Leahy, C. J. Sreenan, and J. J. Quinlan, “Beyond throughput, the next generation: A 5G dataset with channel and context metrics,” in *Proc. ACM Multimedia Systems Conf. (MMSys)*, 2020.
- [18] A. Narayanan *et al.*, “Lumos5G: Mapping and predicting commercial mmWave 5G throughput,” in *Proc. ACM Internet Measurement Conf. (IMC)*, 2020.
- [19] T.-Y. Lin *et al.*, “Microsoft COCO: Common objects in context,” in *Proc. Eur. Conf. Computer Vision (ECCV)*, 2014, pp. 740–755.
- [20] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [21] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [22] R. Koenker and G. Bassett, “Regression quantiles,” *Econometrica*, vol. 46, no. 1, pp. 33–50, 1978.
- [23] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.